AI-DRIVEN AUTOMATED VIDEO GENERATOR FOR CONTENT CREATION

An Application Development-II Report Submitted

In partial fulfillment of the requirement for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

by

DURISHETTY ANIRUDH	**22N31A6648**
BETHAM VIGNESH REDDY	**22N31A6626**
B. RANJITH KUMAR	**22N31A6622**

Under the esteemed Guidance of

Dr. L. Melinda

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY
(Autonomous Institution – UGC, Govt. of India)

(Affiliated to JNTU, Hyderabad, Approved by AICTE, Accredited by NBA & NAAC – 'A' Grade, ISO 9001:2015 Certified) Maisammaguda (v), Near Dullapally, Via: Kompally, Hyderabad – 500 100, Telangana State, India. website: www.mrcet.ac.in

2024-2025

DECLARATION

I hereby declare that the project entitled **"PI-Driven Automated Video Generator for Content Creation"** submitted to **Malla Reddy College of Engineering and Technology,** affiliated t**o** Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering- Artificial Intelligence and Machine Learning** is a result of original research work done by me.

It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

Durishetty Anirudh (22N31A6648)
Betham Vignesh Reddy (22N35A6626)
B. Ranjith Kumar (22N31A6622)

MALLA REDDY COLLEGE
OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS INSTITUTION - UGC, GOVT. OF INDIA)
Affiliated to JNTUH; Approved by AICTE, NBA-Tier 1 & NAAC with A-GRADE | ISO 9001:2015

CERTIFICATE

This is to certify that this is the bonafide record of the project titled **"AI-Driven Automated Video Generator for Content Creation"** submitted by **Durishetty Anirudh** (22N31A6648), **Betham Vignesh Reddy** (22N31A6606), **B. Ranjith Kumar** (22N31A6612) of B.Tech in the partial fulfillment of the requirements for the degree of **Bachelor of Technology** in **Computer Science and Engineering - Artificial Intelligenceand Machine Learning**, Dept. of CI during the year 2024-2025. The results embodied in thisproject report have not been submitted to any other university or institute for the award of anydegree or diploma.

Dr. L. Melinda
Assistant Professor
INTERNAL GUIDE

Dr. D. Sujatha
Professor and Dean (CSE & ET)
HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

Date of Viva-Voce Examination held on: _____

ACKNOWLEDGEMENT

We feel honored and privileged to place our warm salutation to our college Malla Reddy College of Engineering and technology (UGC-Autonomous), our Director ***Dr. VSK Reddy*** who gave us the opportunity to have experience in engineering and profound technical knowledge.

We are indebted to our Principal ***Dr. S. Srinivasa Rao*** for providing us with facilities to do our project and his constant encouragement and moral support which motivated us to move forward with the project.

We would like to express our gratitude to our Head of the Department ***Dr. D. Sujatha,*** Professor and Dean (CSE&ET) for encouraging us in every aspect of our system development and helping us realize our full potential.

We would like to express our sincere gratitude and indebtedness to our project supervisor ***Dr. L. Melinda,*** Assistant Professor for her valuable suggestions and interest throughout the course of this project.

We convey our heartfelt thanks to our Project Coordinator***, Mr. T Hari Babu,*** Assistant Professor for allowing for his regular guidance and constant encouragement during our dissertation work

We would also like to thank all supporting staff of department of CI and all other departments who have been helpful directly or indirectly in making our Application Development-II a success.

We would like to thank our parents and friends who have helped us with their valuable suggestions and support has been very helpful in various phases of the completion of the Application Development-II.

Durishetty Anirudh (22N31A6644)

Betham Vignesh Reddy (22N35A6606)

B. Ranjith Kumar (22N31A6612)

ABSTRACT

This project presents an innovative solution to the challenges of content creation in the rapidly evolving social media landscape. Traditional methods of video production are often manual, time-consuming, and require multiple platforms for scriptwriting, voiceovers, captioning, and video editing. To address these issues, we have developed a fully automated video generation system that streamlines the process from keyword-based script generation to final video rendering. The system integrates several advanced technologies, including ChatGPT for automatic script generation, EdgeTTS for realistic voiceovers, and Whisper for generating captions. Python-based video processing tools are used to combine these elements into a polished final video. The entire process is made accessible through a user-friendly Streamlit interface, which allows users to customize their video outputs with minimal effort. This automated approach enhances efficiency and offers significant time savings for content creators, while maintaining flexibility for customization based on individual needs. The system's seamless integration of cutting-edge technologies provides a comprehensive solution for effortless content creation in today’s fast-paced digital environment.

TABLE OF CONTENTS

1. INTRODUCTION 1

1.1. Problem Statement 1

1.2. Objectives 2

2. LITERATURE SURVEY 3

2.1. Existing System 3

2.2. Proposed System 4

3. SYSTEM REQUIREMENTS 5

3.1. Software and Hardware Requirement 5

3.2. Functional and Non-Functional Requirements 6

3.3. Other Requirements 7

4. SYSTEM DESIGN 9

4.1. Architecture Diagram 9

4.2. UML Diagrams / DFD 11

5. IMPLEMENTATION 15

5.1. Algorithms 15

5.2. Architectural Components 41

5.3. Feature Extraction 42

5.4. Packages/Libraries Used 43

5.5. Output Screens 44

6. SYSTEM TESTING 50

6.1. Test Cases 50

6.2. Results and Discussions 52

6.3. Performance Evaluation 55

7. CONCLUSION & FUTURE ENHANCEMENTS 56

8. REFERENCES 58

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

In today's digital era, video content has become a powerful medium for communication, learning, storytelling, and brand engagement. However, **small content creators, educators, and independent professionals** often struggle to keep up with the growing demand for high-quality video content. Unlike large media houses and production studios, these individuals lack access to skilled manpower, expensive tools, and the time-intensive workflows typically associated with professional video creation.

The traditional video production pipeline — involving **scriptwriting, voiceover recording, caption generation, editing, and rendering** — is highly fragmented and often requires switching between multiple software platforms. This process can be daunting and time-consuming, especially for creators with limited technical expertise or resources.

As a result, many promising ideas never materialize into engaging videos, or are executed with inconsistent quality. This creates a significant barrier for creators who wish to share knowledge, promote products, or entertain audiences without being overwhelmed by the production overhead. Therefore, there is a strong need for an **automated, intelligent, and user-friendly solution** that simplifies and accelerates the video creation process while maintaining professional quality.

1.2 Objectives

The primary objective of this Software Requirements Specification (SRS) document is to define the **functional and non-functional requirements** for the **AI-Driven Automated Video Generator** — a system designed to **fully automate the video creation process** using state-of-the-art Artificial Intelligence technologies.

The system aims to empower users by offering an **end-to-end, streamlined video generation experience**, allowing them to convert simple textual input into fully rendered video content with minimal effort. The core objectives include:

- **Automated Script Generation**: Transforming user-provided topics or keywords into structured and engaging video scripts using natural language models like ChatGPT.
- **Voiceover Synthesis**: Converting scripts into realistic, human-like speech using neural TTS (Text-to-Speech) technology with multiple voice options.
- **Caption Generation**: Automatically generating and synchronizing subtitles using speech-to-text models to enhance accessibility and engagement.
- **Video Assembly**: Searching for relevant background videos using AI-generated keywords and compiling them with audio and captions into a final rendered video.

This SRS also aims to document the system's **technical architecture, performance benchmarks, design constraints, user interaction flows, legal considerations, and implementation guidelines**, providing a clear development blueprint for all stakeholders including developers, testers, and supervisors.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing System:

Currently, video content creation is heavily dependent on manual intervention, with most creators using a combination of tools for scriptwriting, voiceover recording, video editing, and caption synchronization. Some existing systems include:

- **Traditional Methods:**
  - Manual scriptwriting and research.
  - Hiring voice artists for narration.
  - Using software like Adobe Premiere Pro or Final Cut Pro for video editing.
  - Manually adding captions using video editing tools.
- **AI-Assisted Solutions:**
  - Some platforms offer text-to-speech conversions but lack video integration.
  - Certain AI-driven tools generate captions but do not sync them with videos effectively.
  - Existing solutions do not provide full automation from script generation to video rendering with minimalistic approach.

2.1.1 Drawbacks of existing system:

The existing approaches to video production come with several challenges:

- **Time-Consuming Workflow:** Creating a single professional-quality video can take hours or even days due to manual effort at multiple stages.
- **Fragmented Toolsets:** Users rely on multiple applications for scriptwriting, voiceovers, video editing, and captioning, leading to inefficiencies.
- **High Costs:** Professional content creation tools and hiring voice artists add significant expenses to the production process.
- **Inconsistent Quality:** Manually created content often lacks uniformity, affecting branding and engagement consistency.

2.2 Proposed System:

- To address these limitations, the AI-Driven Automated Video Generator offers a **fully integrated and automated solution**:
- **AI-Powered Scriptwriting:** Generates topic-based scripts using **ChatGPT**, ensuring engaging and well-structured content.
- **Realistic Voiceover Synthesis:** Uses **EdgeTTS** to produce human-like narration with multiple voice and language options.
- **Accurate Captioning:** Implements **Whisper** to create perfectly synchronized captions for accessibility and engagement.
- **Automated Video Retrieval:** Searches and selects suitable background footage from **Pexels API** based on AI-analyzed keywords.
- **One-Click Video Rendering:** Combines all elements using **MoviePy and FFmpeg**, producing high-quality MP4 videos ready for sharing.
- **Customizability:** Allows users to edit generated scripts, select different voiceovers, adjust captions, and preview the video before finalization.
- By combining AI-based automation with a user-friendly interface, the system significantly reduces video production time while ensuring high-quality output at minimal cost.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 Software and Hardware Requirements

Software Requirements:

- **Operating System:** Compatible with Windows 10/11, Linux, and macOS.
- **Programming Language:** Python 3.11.
- **Web Framework:** Streamlit for UI and API interactions.
- **Libraries and Dependencies:** OpenAI API, EdgeTTS, Whisper, MoviePy, FFmpeg, NumPy, Pandas.
- **Cloud and Storage:** AWS or Google Drive for storing generated videos.
- **Version Control:** GitHub for source code management and collaboration.
- **Database:** Log storage and retrieval mechanisms for AI-generated scripts, captions, and metadata.

Hardware Requirements:

- **Processor:** Ryzen 7 5800H.
- **RAM:** Minimum 16GB (Recommended 32GB for high-performance tasks).
- **Storage:** 20GB free space (SSD preferred for faster data access).
- **GPU:** NVIDIA RTX 3050 4GB or equivalent for AI model acceleration.
- **Internet Connection:** High-speed internet for real-time API calls and media processing.

3.2 Functional and Non-Functional Requirements

- **Functional:**
    - Generate scripts
    - Synthesize voiceovers
    - Create captions
    - Compile videos seamlessly
- **Non-Functional:**
    - Efficient (implying a requirement for speed or performance)
    - Scalable (implying a requirement for flexibility and adaptability)
    - Accessible (implying a requirement for ease of use)

3.3 Other Requirements

1. **User Input Handling**
    - The system must allow users to input a topic or short description to initiate the script generation.
2. **Script Generation**
    - The system must generate a well-structured script using the OpenAI ChatGPT API.
3. **Voiceover Generation**
    - The system must convert the generated script into audio using EdgeTTS with options for selecting voice, gender, and accent.
4. **Caption Generation**
    - The system must extract timed captions from the generated voiceover using the Whisper model.
5. **Video Search**
    - The system must extract keywords from the script and captions to generate video search queries.
6. **Background Video Retrieval**
    - The system must use the Pexels API to fetch video clips that match the generated search queries.
7. **Video Assembly**
    - The system must combine audio, captions, and video clips into a complete, synchronized video using MoviePy and FFmpeg.
8. **Preview and Download**
    - The system must allow users to preview the generated video and download it in MP4 format.
9. **Customizability**
    - The system must allow users to edit scripts, change voices, re-generate voiceovers, and upload custom videos.
10. **Stage-wise Navigation**
    - The system must support seamless navigation between stages (Script → Voiceover → Video Generation).
11. **Performance Requirements**

- Script generation should complete within 3 seconds.
- Voiceover synthesis should complete within 5 seconds for a 1-minute script.
- Full video rendering should complete within 2 minutes.

12. **Scalability**
    - The system shall support batch processing and handle multiple video generation requests simultaneously.
13. **Usability**
    - The interface shall be simple, intuitive, and usable by non-technical users.
14. **Portability**
    - The system shall run on Windows, Linux, and macOS environments with minimal setup.
15. **Security**
    - All user data and media files shall be handled securely, and generated outputs should comply with copyright regulations.
16. **Maintainability**
    - The codebase shall follow modular structure and be version-controlled using GitHub for easier updates and debugging.
17. **Reliability**
    - The system shall produce consistent outputs across different runs with the same input.
18. **Extensibility**
    - The platform shall support easy integration of additional features such as multilingual support, more voice styles, or new video APIs.
19. **Legal & Ethical Compliance**
    - The system shall only use royalty-free assets and avoid generating offensive or misleading content.
20. **Documentation**
21. The system shall be accompanied by user guides, code documentation, and architectural diagrams for ease of use and development.

CHAPTER-4
SYSTEM DESIGN

4.1 System Architecture

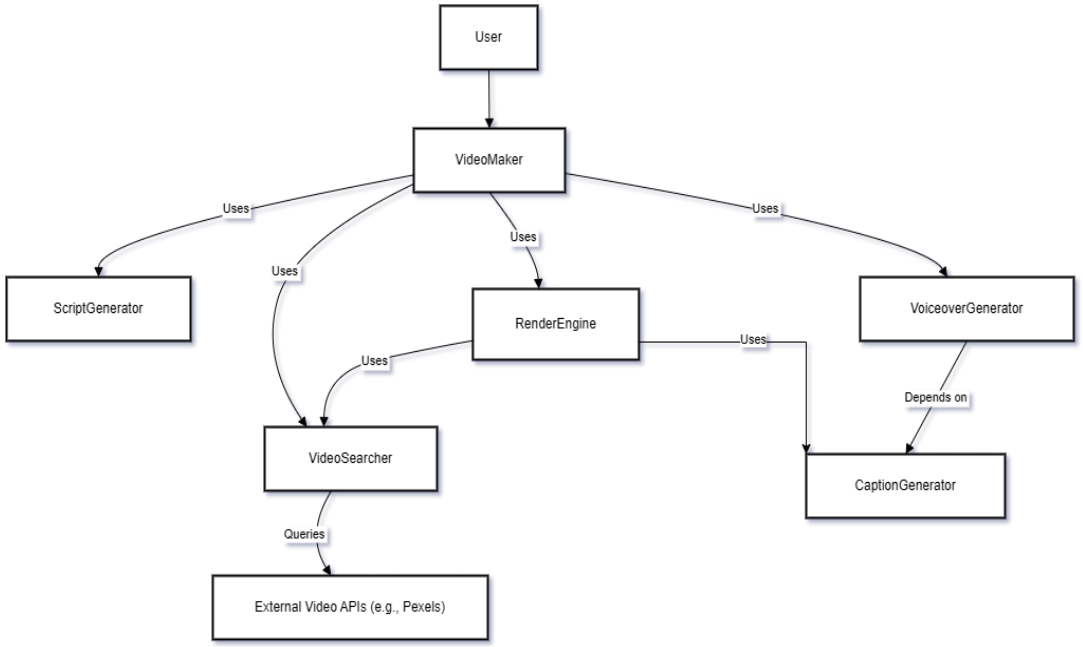


Fig 4.1 System Architecture of AI-Driven Automated Video Generator for Content Creation

4.2 Data flow diagram

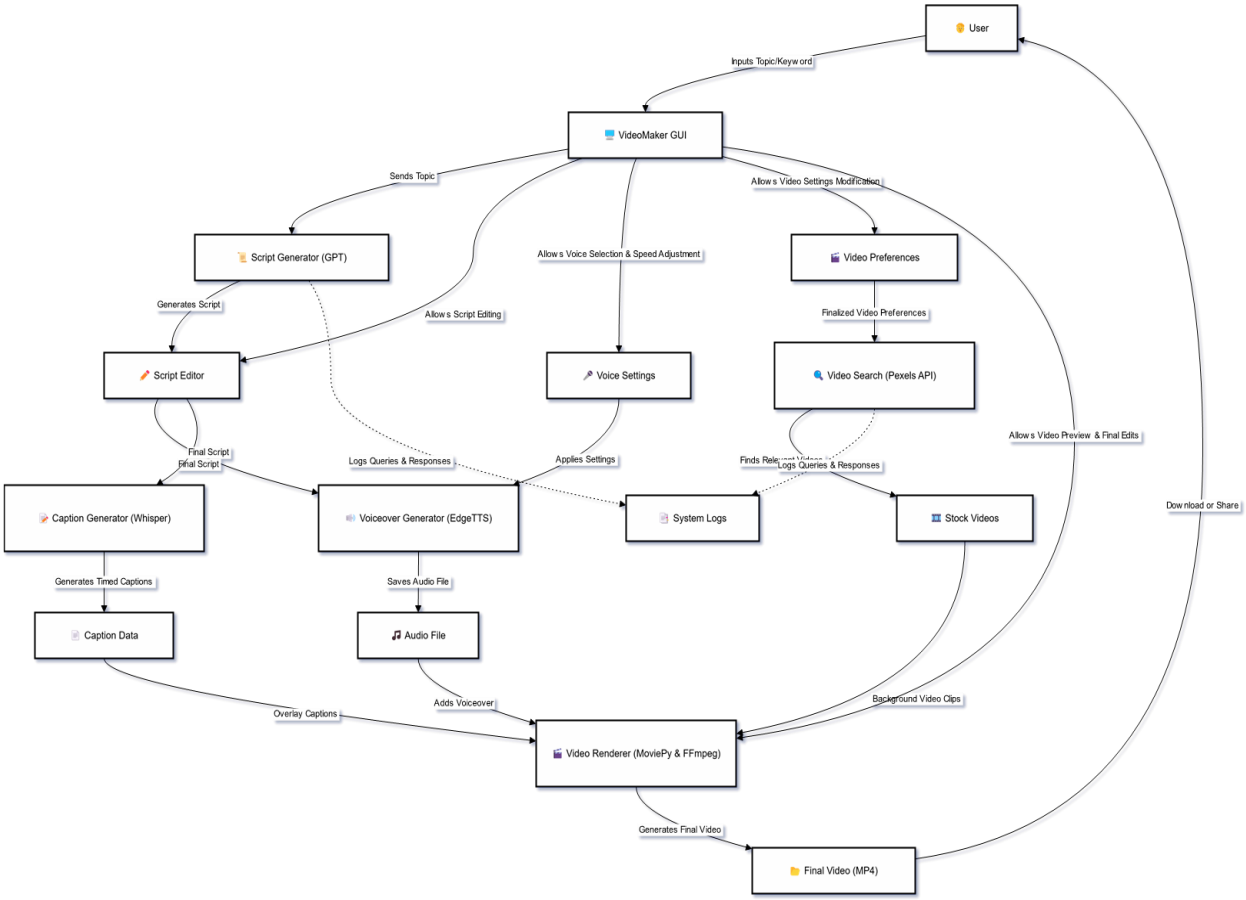


Fig 4.2 Data Flow Diagram for AI-Driven Automated Video Generator for Content Creation

4.3 UML Diagrams

The unified modeling is a standard language for specifying, visualizing, constructing and documenting the system and its components is a graphical language which provides a vocabulary and set of semantics and rules. The UML focuses on the conceptual and physical representation of the system. It captures the decisions and understandings about systems that must be constructed. It is used to understand, design, configure and control information about the systems.

4.3.1 Use case diagram

A use case diagram is a graph of actors set of use cases enclosed by a system boundary, communication associations between the actors and users and generalization among use cases. The use case model defines the outside (actors) and inside (use case) of the system's behavior.

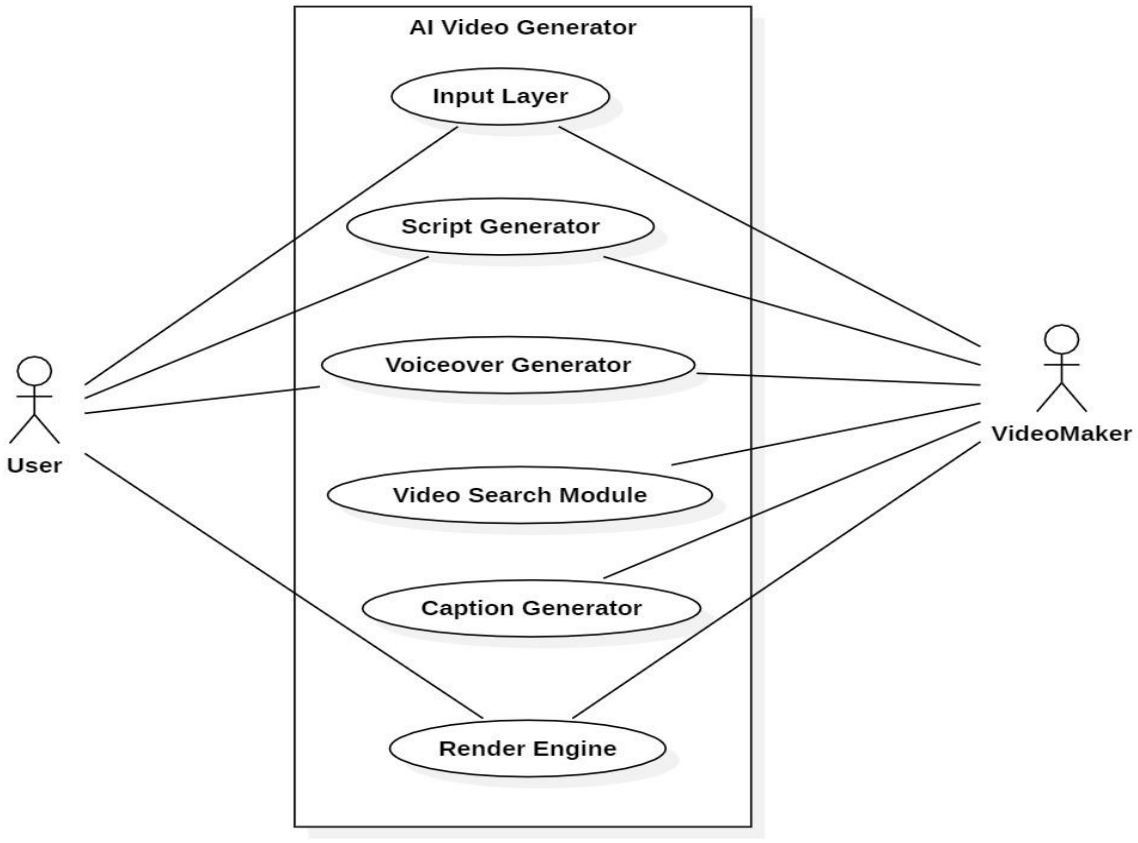


Fig 4.3.1 Use Case Diagram for AI-Driven Automated Video Generator for Content Creation

4.3.2 Class Diagram

A class is a representation of an object and, in many ways; it is simply a template from which objects are created. Classes form the main building blocks of an object-oriented application. Although thousands of students attend the university, you would only model one class, called Student, which would represent the represent the entire collection of students.

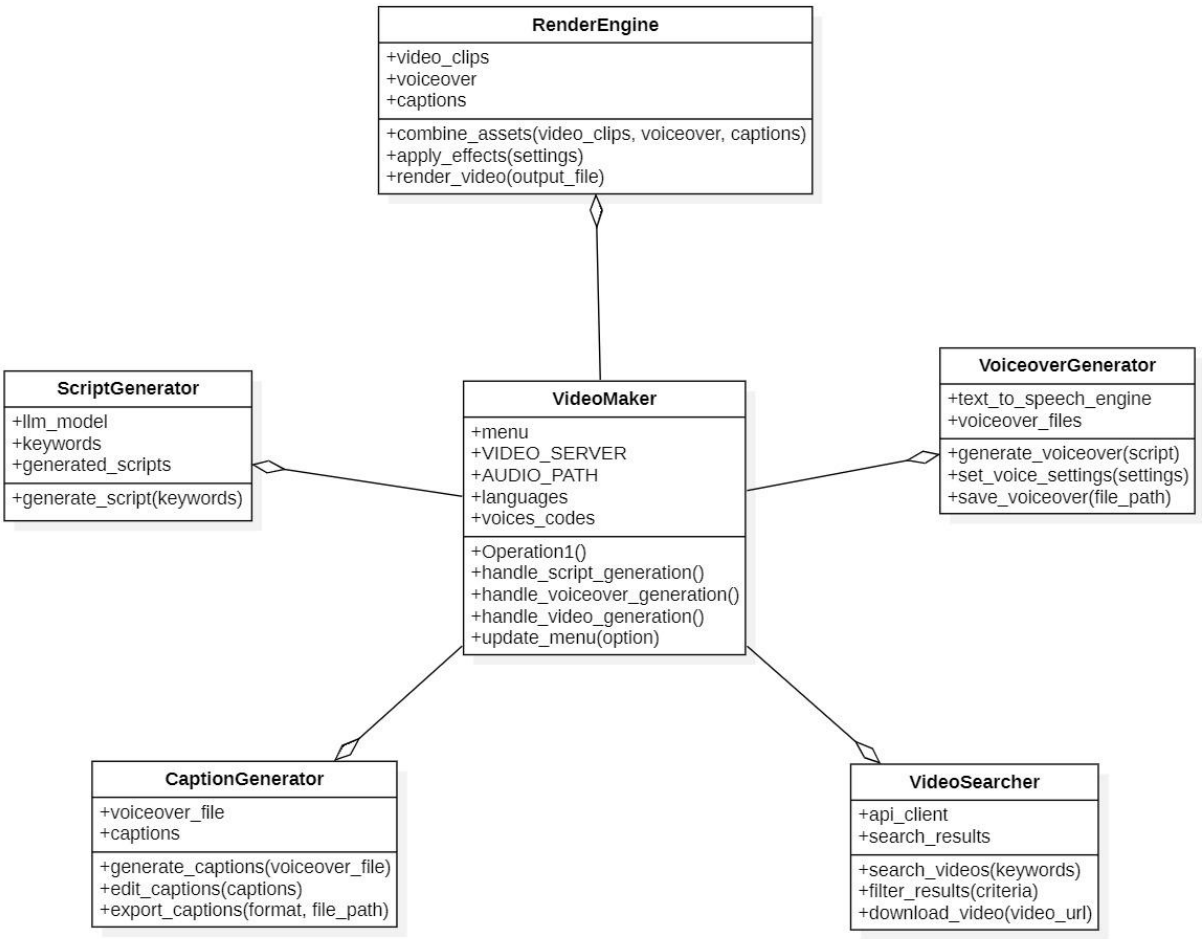


Fig 4.3.2 Class Diagram for AI-Driven Automated Video Generator for Content Creation

4.3.3 Sequence Diagram

Sequence diagram is used to represent the flow of messages, events and actions between the objects or components of a system. Time is represented in the vertical direction showing the sequence of interactions of the header elements, which are displayed horizontally at the top of the diagram.

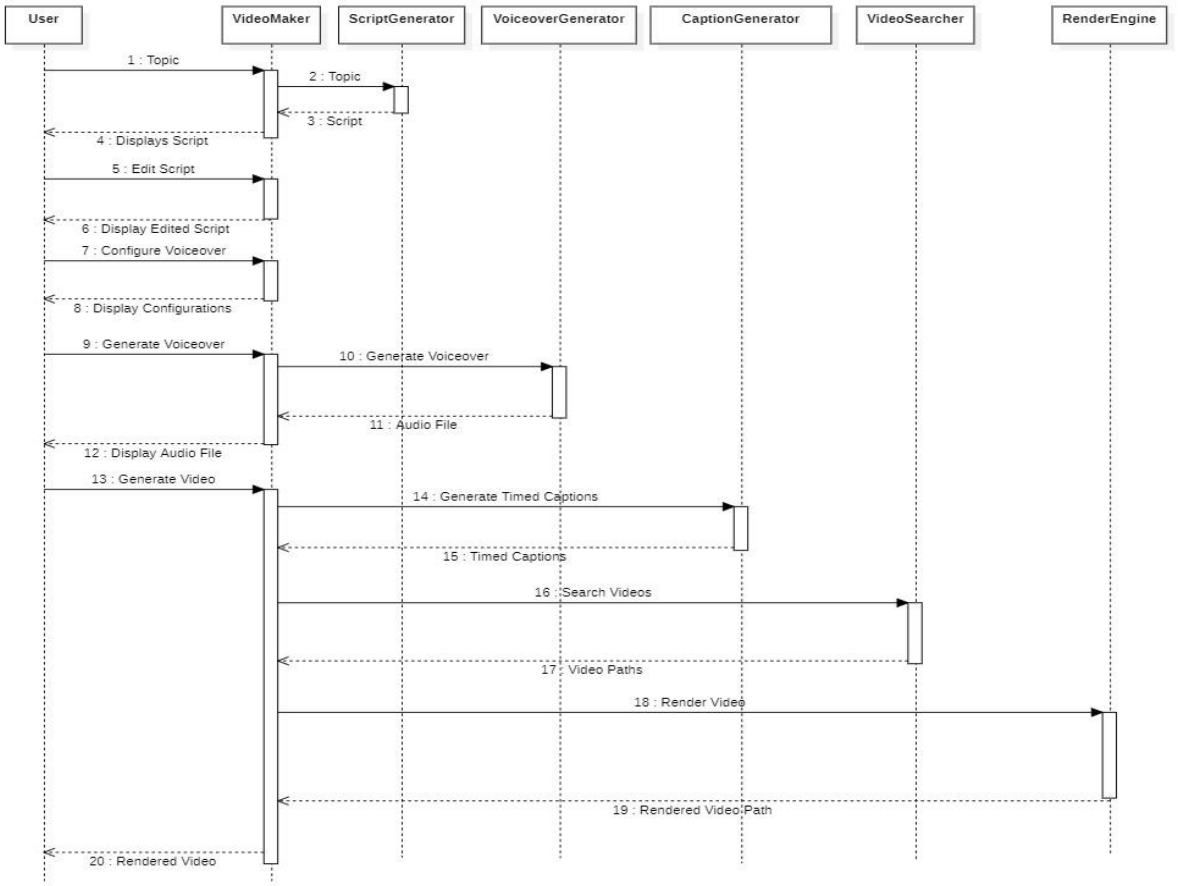
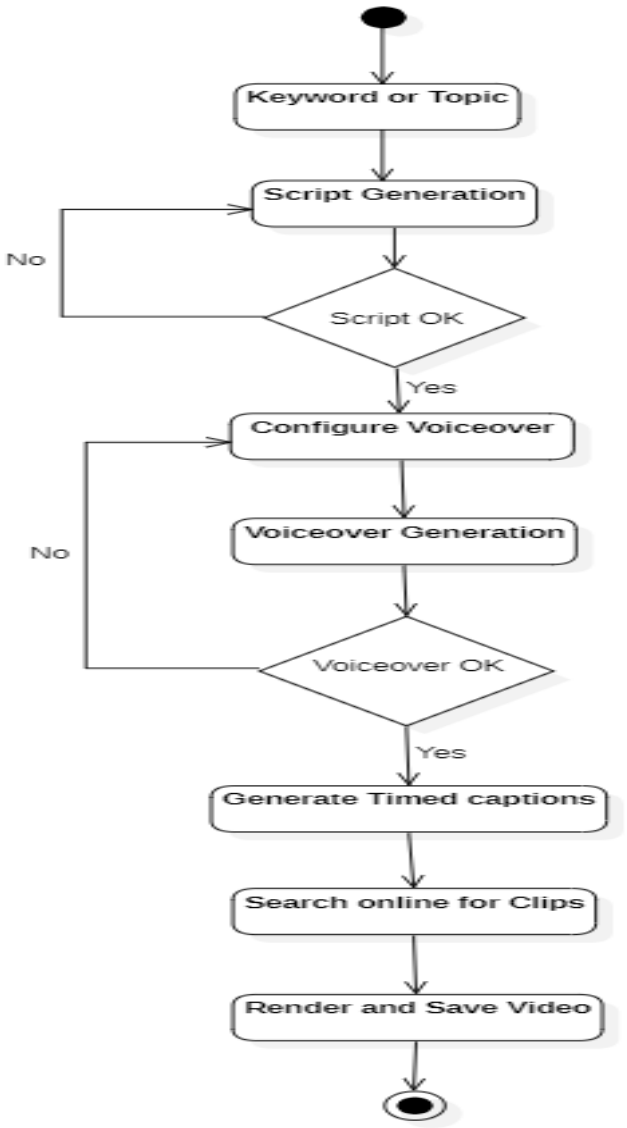


Fig 4.3.3 Activity Diagram for AI-Driven Automated Video Generator for Content Creation

4.3.4 Activity Diagram

Activity diagram represents the business and operational workflows of a system. An Activity diagram is a dynamic diagram that shows the activity and the event that causes the object



to be in the particular state.

Fig 4.3.4 Activity Diagram for AI-Driven Automated Video Generator for Content Creation

CHAPTER-5
IMPLEMENTATION

5.1 Algorithms

The AI-Driven Automated Video Generator follows a step-by-step process powered by intelligent algorithms:

1. **Script Generation**: The system uses a language model (ChatGPT) to create a meaningful script from a user-provided topic.
2. **Voiceover Creation**: The script is converted into speech using EdgeTTS, with options for different voices and accents.
3. **Caption Generation**: Whisper processes the audio to create accurate, time-synced captions.
4. **Video Clip Search**: The script and captions are analyzed to generate search terms, which are used to find relevant video clips from the Pexels API.
5. **Clip Matching and Merging**: Retrieved video clips are matched with caption timings. If gaps exist, a simple merging technique is used to fill them.
6. **Final Video Rendering**: All components — audio, video, and captions — are combined using MoviePy and FFmpeg to produce the final MP4 video.

5.1.1 Code

VideoMaker.py

```
import streamlit as st
import Passwords  # Assuming you need this elsewhere
import os
from utility.captions.timed_captions_generator import generate_timed_captions
from utility.render.render_engine import get_output_media
from utility.video.background_video_generator import generate_video_url
from utility.video.video_search_query_generator import getVideoSearchQueriesTimed,
merge_empty_intervals  # Assuming you need this elsewhere
st.set_page_config(layout="wide")
st.sidebar.subheader = "Stage"
if "menu" not in st.session_state:
    st.session_state["menu"] = "Script Generation"
menu = st.sidebar.selectbox(
```

```
    "",
    ["Script Generation", "Voiceover", "Video Generation"],
    index=["Script Generation", "Voiceover", "Video Generation"].index(st.session_state["menu"])
)
VIDEO_SERVER = "pexel"
AUDIO_PATH = "audio_tts.wav"
languages = ["English (United States)",
             "English (United Kingdom)",
             "English (Australia)",
             "English (Canada)",
             "English (India)",
             "English (New Zealand)",
             "English (Ireland)",
             "English (South Africa)"
             ]
voices_codes = {
    "English (United States)": [
        ["en-US-GuyNeural", "en-US-ChristopherNeural"],  # Male voices
        ["en-US-AriaNeural", "en-US-JennyNeural", "en-US-AmberNeural", "en-US-AnaNeural"]  # Female voices
    ],
    "English (United Kingdom)": [
        ["en-GB-RyanNeural", "en-GB-GeorgeNeural"],  # Male voices
        ["en-GB-LibbyNeural", "en-GB-SoniaNeural"]  # Female voices
    ],
    "English (Australia)": [
        ["en-AU-WilliamNeural"],  # Male voices
        ["en-AU-NatashaNeural"]  # Female voices
    ],
    "English (Canada)": [
        ["en-CA-LiamNeural"],  # Male voices
```

```
        ["en-CA-ClaraNeural"]  # Female voices
    ],
    "English (India)": [
        ["en-IN-PrabhatNeural"],  # Male voices
        ["en-IN-NeerjaNeural"]  # Female voices
    ],
    "English (New Zealand)": [
        ["en-NZ-MitchellNeural"],  # Male voices
        ["en-NZ-MollyNeural"]  # Female voices
    ],
    "English (Ireland)": [
        ["en-IE-ConnorNeural"],  # Male voices
        ["en-IE-EmilyNeural"]  # Female voices
    ],
    "English (South Africa)": [
        ["en-ZA-ThembaNeural"],  # Male voices
        ["en-ZA-TessaNeural"]  # Female voices
    ]
}
if menu == "Script Generation":
    st.title("Script Generation and Voiceover")
    st.markdown(
        """
        <style>
        .custom-label {
            font-size: 20px;
            font-weight: bold;
        }
        </style>
        """,
        unsafe_allow_html=True,)
```

```
# Initialize session state variables
if "textarea_value" not in st.session_state:
    st.session_state["textarea_value"] = ""  # Default value for script
if "audio" not in st.session_state:
    st.session_state["audio"] = 0 # Default value for
# Function to generate the script
def generate_script(user_prompt):
    from utility.script.script_generator import generate_script
    return generate_script(user_prompt)
# Create columns for input and button
col1, col2 = st.columns([4, 1])  # Adjust column widths as needed
# Capture user prompt
with col1:
    user_prompt = st.text_input(
        "Topic",
        placeholder="Enter your topic here"
    )
# Add button to generate script
with col2:
    st.markdown("<br>", unsafe_allow_html=True)  # Add space to align with text input
    if st.button("Generate Script"):
        if user_prompt.strip():  # Ensure the input is not empty
            st.session_state["textarea_value"] = generate_script(user_prompt)
        else:
            st.warning("Please enter a valid topic to generate a script.")
# Render the updated text area value
script = st.text_area("Script", height=300, value=st.session_state["textarea_value"])
if st.session_state["textarea_value"] != "":
    def script_next():
        st.session_state["menu"] = "Voiceover"
    st.button("Next",on_click=script_next)
```

```
if menu == "Voiceover" :
    if st.session_state["textarea_value"] != "":
        # script = "Get ready for some incredible turtle facts! 1. Turtles have been around for 220 million years, surviving alongside dinosaurs. 2. They can't retract their limbs into their shells for protection like other turtles might. 3. Sea turtles can travel up to 22 miles per hour, faster than some cars! 4. Leatherback sea turtles are the largest turtle species, sometimes growing over 6 feet long. 5. A turtle's shell is not just one piece; it's made up of over 50 bones. 6. Despite their slow speed, turtles can live for over 100 years, making them some of the longest-living creatures. 7. Turtle eggs can take up to 100 days to hatch! 8. Turtles play a key role in the ecosystem by controlling jellyfish and sea grass populations. Fascinating, huh?"
        st.title("Voiceover Generation")
        dash, configuration = st.columns([70,30])
        dash.markdown(
            """
            <h3 style="margin-bottom: 10px;">Your script</h3>
            """,
            unsafe_allow_html=True,
        )
        # Display the script
        dash.markdown(
            f"""
            <div style="margin-top: 0px; font-size: 16px; line-height: 1.5; text-align:justify;">
                {st.session_state["textarea_value"]}
            </div>
            """,
            unsafe_allow_html=True,
        )
        col1, col2= configuration.columns([1,3])
        col2.subheader("Audio Settings")
        lang = col2.selectbox("Select Language", languages)
        col2.markdown("<br>", unsafe_allow_html=True)
```

```
        gender = col2.selectbox("Select Voice Gender",["Male", "Female"])
        col2.markdown("<br>", unsafe_allow_html=True)
         if gender == "Male":
            voices = voices_codes[lang][0]
        else:
            voices = voices_codes[lang][1]
        voice = col2.selectbox("Select Voice Gender", voices)
        col2.markdown("<br>", unsafe_allow_html=True)
        if col2.button("Generate Voiceover"):
                import asyncio
                from utility.audio.audio_generator import generate_audio
                asyncio.run(generate_audio(st.session_state["textarea_value"], AUDIO_PATH, voice))
                # asyncio.run(generate_audio(script, AUDIO_PATH, voice))
                st.session_state["audio"] = 1
        dash.markdown("<br>", unsafe_allow_html=True)
        if os.path.exists(AUDIO_PATH) and st.session_state["audio"] == 1:
            dash.subheader("Voicover Audio")
            dash.audio(AUDIO_PATH)
        col1,col2,col3,col4,col5,col6,col7 = dash.columns(7)
        def voice_prev():
            st.session_state["menu"] = "Script Generation"
        col1.button("Previous", on_click= voice_prev)
        if os.path.exists(AUDIO_PATH) and st.session_state["audio"] == 1:
            def voice_next():
                st.session_state["menu"] = "Video Generation"
            col2.button("Next",on_click=voice_next)
    else:
        st.title("Voiceover Generation")
        st.warning("Please complete the Script Generation and you will be able to generate audio here.
")
if menu == "Video Generation":
```

```
    st.title("Video Generation")
    if st.session_state["audio"] == 1 and st.session_state["textarea_value"] != "":
        st.markdown("All Set! Ready for the magic?")
        def generate_video():
            timed_captions = generate_timed_captions(AUDIO_PATH)
            print(timed_captions)
            search_terms = getVideoSearchQueriesTimed(st.session_state["textarea_value"],
timed_captions)
            print(search_terms)
            background_video_urls = None
            if search_terms is not None:
                background_video_urls = generate_video_url(search_terms, VIDEO_SERVER)
                print(background_video_urls)
            else:
                print("No background video")
            background_video_urls = merge_empty_intervals(background_video_urls)
            if background_video_urls is not None:
                video = get_output_media(AUDIO_PATH, timed_captions, background_video_urls,
VIDEO_SERVER)
                print(video)
        st.button("Next",on_click=generate_video)
        if os.path.exists("rendered_video.mp4"):
            st.video("rendered_video.mp4")
    else:
        st.warning("Please complete the Script Generation and Voiceover Generation to access Video
Generation")
```

audio_generator.py

```
import edge_tts

async def generate_audio(text,outputFilename, voice):
    communicate = edge_tts.Communicate(text,voice)
    await communicate.save(outputFilename)
```

timed_captions_generator.py

```
import whisper_timestamped as whisper
from whisper_timestamped import load_model, transcribe_timestamped
import re

def generate_timed_captions(audio_filename,model_size="base"):
    WHISPER_MODEL = load_model(model_size)

    gen = transcribe_timestamped(WHISPER_MODEL, audio_filename, verbose=False, fp16=False)

    return getCaptionsWithTime(gen)

def splitWordsBySize(words, maxCaptionSize):

    halfCaptionSize = maxCaptionSize / 2
    captions = []
    while words:
        caption = words[0]
        words = words[1:]
        while words and len(caption + ' ' + words[0]) <= maxCaptionSize:
            caption += ' ' + words[0]
            words = words[1:]
            if len(caption) >= halfCaptionSize and words:
                break
```

```
        captions.append(caption)
    return captions

def getTimestampMapping(whisper_analysis):

    index = 0
    locationToTimestamp = {}
    for segment in whisper_analysis['segments']:
        for word in segment['words']:
            newIndex = index + len(word['text'])+1
            locationToTimestamp[(index, newIndex)] = word['end']
            index = newIndex
    return locationToTimestamp

def cleanWord(word):

    return re.sub(r'[^\w\s\-_"\'\']', '', word)

def interpolateTimeFromDict(word_position, d):

    for key, value in d.items():
        if key[0] <= word_position <= key[1]:
            return value
    return None

def getCaptionsWithTime(whisper_analysis, maxCaptionSize=15, considerPunctuation=False):

    wordLocationToTime = getTimestampMapping(whisper_analysis)
    position = 0
    start_time = 0
    CaptionsPairs = []
```

```
    text = whisper_analysis['text']

    if considerPunctuation:
        sentences = re.split(r'(?<=[.!?]) +', text)
            words = [word for sentence in sentences for word in splitWordsBySize(sentence.split(), maxCaptionSize)]
    else:
        words = text.split()
        words = [cleanWord(word) for word in splitWordsBySize(words, maxCaptionSize)]

    for word in words:
        position += len(word) + 1
        end_time = interpolateTimeFromDict(position, wordLocationToTime)
        if end_time and word:
            CaptionsPairs.append(((start_time, end_time), word))
            start_time = end_time

    return CaptionsPairs
```

render_engine.py

```
import time
import os
import tempfile
import zipfile
import platform
import subprocess
from moviepy.editor import (AudioFileClip, CompositeVideoClip, CompositeAudioClip, ImageClip,
                            TextClip, VideoFileClip)
from moviepy.audio.fx.audio_loop import audio_loop
from moviepy.audio.fx.audio_normalize import audio_normalize
import requests

def download_file(url, filename):
    with open(filename, 'wb') as f:
        headers = {
         "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/91.0.4472.124 Safari/537.36"
        }
        response = requests.get(url, headers=headers)
        f.write(response.content)

def search_program(program_name):
    try:
        search_cmd = "where" if platform.system() == "Windows" else "which"
        return subprocess.check_output([search_cmd, program_name]).decode().strip()
    except subprocess.CalledProcessError:
        return None

def get_program_path(program_name):
    program_path = search_program(program_name)
```

```
    return program_path

def get_output_media(audio_file_path, timed_captions, background_video_data, video_server):
    OUTPUT_FILE_NAME = "rendered_video.mp4"
    magick_path = get_program_path("magick")
    print(magick_path)
    if magick_path:
        os.environ['IMAGEMAGICK_BINARY'] = magick_path
    else:
        os.environ['IMAGEMAGICK_BINARY'] = '/usr/bin/convert'

    visual_clips = []
    for (t1, t2), video_url in background_video_data:
        # Download the video file
        video_filename = tempfile.NamedTemporaryFile(delete=False).name
        download_file(video_url, video_filename)

        # Create VideoFileClip from the downloaded file
        video_clip = VideoFileClip(video_filename)
        video_clip = video_clip.set_start(t1)
        video_clip = video_clip.set_end(t2)
        visual_clips.append(video_clip)

    audio_clips = []
    audio_file_clip = AudioFileClip(audio_file_path)
    audio_clips.append(audio_file_clip)

    for (t1, t2), text in timed_captions:
                text_clip = TextClip(txt=text, fontsize=100, color="white", stroke_width=3,
stroke_color="black", method="label")
        text_clip = text_clip.set_start(t1)
```

```
        text_clip = text_clip.set_end(t2)
        text_clip = text_clip.set_position(["center", 800])
        visual_clips.append(text_clip)

    video = CompositeVideoClip(visual_clips)

    if audio_clips:
        audio = CompositeAudioClip(audio_clips)
        video.duration = audio.duration
        video.audio = audio

        video.write_videofile(OUTPUT_FILE_NAME, codec='libx264', audio_codec='aac', fps=25, preset='veryfast')

    # Clean up downloaded files
    for (t1, t2), video_url in background_video_data:
        video_filename = tempfile.NamedTemporaryFile(delete=False).name
        os.remove(video_filename)

    return OUTPUT_FILE_NAME
```

script_generator.py

```
import os
from openai import OpenAI
import json

if len(os.environ.get("GROQ_API_KEY")) > 30:
    from groq import Groq
    model = "llama-3.3-70b-versatile"
    client = Groq(
        api_key=os.environ.get("GROQ_API_KEY"),
        )
else:
    OPENAI_API_KEY = os.getenv('OPENAI_KEY')
    model = "gpt-4o"
    client = OpenAI(api_key=OPENAI_API_KEY)

def generate_script(topic):
    prompt = (
        """You are a seasoned content writer for a YouTube Shorts channel, specializing in facts videos.
        Your facts shorts are concise, each lasting less than 50 seconds (approximately 140 words).
         They are incredibly engaging and original. When a user requests a specific type of facts short, you will create it.

        For instance, if the user asks for:
        Weird facts
        You would produce content like this:

        Weird facts you don't know:
        - Bananas are berries, but strawberries aren't.
        - A single cloud can weigh over a million pounds.
        - There's a species of jellyfish that is biologically immortal.
```

- Honey never spoils; archaeologists have found pots of honey in ancient Egyptian tombs that are over 3,000 years old and still edible.

- The shortest war in history was between Britain and Zanzibar on August 27, 1896. Zanzibar surrendered after 38 minutes.

- Octopuses have three hearts and blue blood.

You are now tasked with creating the best short script based on the user's requested type of 'facts'.

Keep it brief, highly interesting, and unique.

Stictly output the script in a JSON format like below, and only provide a parsable JSON object with the key 'script'.

```
    # Output
    {"script": "Here is the script ..."}
    """
)

response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": topic}
        ]
    )
content = response.choices[0].message.content
try:
    script = json.loads(content)["script"]
except Exception as e:
    json_start_index = content.find('{')
    json_end_index = content.rfind('}')
```

```
    print(content)
    content = content[json_start_index:json_end_index+1]
    script = json.loads(content)["script"]
return script
```

background_video_generator.py

```
import os
import requests
from utility.utils import log_response,LOG_TYPE_PEXEL

PEXELS_API_KEY = os.environ.get('PEXELS_KEY')

def search_videos(query_string, orientation_landscape=True):

    url = "https://api.pexels.com/videos/search"
    headers = {
        "Authorization": PEXELS_API_KEY,
         "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36"
    }
    params = {
        "query": query_string,
        "orientation": "landscape" if orientation_landscape else "portrait",
        "per_page": 15
    }

    response = requests.get(url, headers=headers, params=params)
    json_data = response.json()
    log_response(LOG_TYPE_PEXEL,query_string,response.json())

    return json_data

def getBestVideo(query_string, orientation_landscape=True, used_vids=[]):
    vids = search_videos(query_string, orientation_landscape)
    videos = vids['videos']  # Extract the videos list from JSON
```

```
    # Filter and extract videos with width and height as 1920x1080 for landscape or 1080x1920 for portrait
    if orientation_landscape:
        filtered_videos = [video for video in videos if video['width'] >= 1920 and video['height'] >= 1080 and video['width']/video['height'] == 16/9]
    else:
        filtered_videos = [video for video in videos if video['width'] >= 1080 and video['height'] >= 1920 and video['height']/video['width'] == 16/9]

    # Sort the filtered videos by duration in ascending order
    sorted_videos = sorted(filtered_videos, key=lambda x: abs(15-int(x['duration'])))

    # Extract the top 3 videos' URLs
    for video in sorted_videos:
        for video_file in video['video_files']:
            if orientation_landscape:
                if video_file['width'] == 1920 and video_file['height'] == 1080:
                    if not (video_file['link'].split('.hd')[0] in used_vids):
                        return video_file['link']
            else:
                if video_file['width'] == 1080 and video_file['height'] == 1920:
                    if not (video_file['link'].split('.hd')[0] in used_vids):
                        return video_file['link']
    print("NO LINKS found for this round of search with query :", query_string)
    return None

def generate_video_url(timed_video_searches,video_server):
        timed_video_urls = []
        if video_server == "pexel":
            used_links = []
            for (t1, t2), search_terms in timed_video_searches:
```

```
        url = ""
        for query in search_terms:

            url = getBestVideo(query, orientation_landscape=True, used_vids=used_links)
            if url:
                used_links.append(url.split('.hd')[0])
                break
        timed_video_urls.append([[t1, t2], url])
elif video_server == "stable_diffusion":
    timed_video_urls = get_images_for_video(timed_video_searches)

return timed_video_urls
```

video_search_query_generator.py

```
from openai import OpenAI
import os
import json
import re
from datetime import datetime
from utility.utils import log_response,LOG_TYPE_GPT

if len(os.environ.get("GROQ_API_KEY")) > 30:
    from groq import Groq
    model = "llama3-70b-8192"
    client = Groq(
        api_key=os.environ.get("GROQ_API_KEY"),
        )
else:
    model = "gpt-4o"
    OPENAI_API_KEY = os.environ.get('OPENAI_KEY')
    client = OpenAI(api_key=OPENAI_API_KEY)

log_directory = ".logs/gpt_logs"

prompt = """# Instructions
```

Given the following video script and timed captions, extract three visually concrete and specific keywords for each time segment that can be used to search for background videos. The keywords should be short and capture the main essence of the sentence. They can be synonyms or related terms. If a caption is vague or general, consider the next timed caption for more context. If a keyword is a single word, try to return a two-word keyword that is visually concrete. If a time frame contains two or more important pieces of information, divide it into shorter time frames with one keyword each. Ensure that the time periods are strictly consecutive and cover the entire length of the video. Each keyword should cover between 2-4 seconds. The output should be in JSON format, like this: [[[t1, t2],

["keyword1", "keyword2", "keyword3"]], [[t2, t3], ["keyword4", "keyword5", "keyword6"]], ...]. Please handle all edge cases, such as overlapping time segments, vague or general captions, and single-word keywords.

For example, if the caption is 'The cheetah is the fastest land animal, capable of running at speeds up to 75 mph', the keywords should include 'cheetah running', 'fastest animal', and '75 mph'. Similarly, for 'The Great Wall of China is one of the most iconic landmarks in the world', the keywords should be 'Great Wall of China', 'iconic landmark', and 'China landmark'.

Important Guidelines:

Use only English in your text queries.
Each search string must depict something visual.
The depictions have to be extremely visually concrete, like rainy street, or cat sleeping.
'emotional moment' <= BAD, because it doesn't depict something visually.
'crying child' <= GOOD, because it depicts something visual.
The list must always contain the most relevant and appropriate query searches.
['Car', 'Car driving', 'Car racing', 'Car parked'] <= BAD, because it's 4 strings.
['Fast car'] <= GOOD, because it's 1 string.
['Un chien', 'une voiture rapide', 'une maison rouge'] <= BAD, because the text query is NOT in English.

Note: Your response should be the response only and no extra text or data.
"""

```
def fix_json(json_str):
    # Replace typographical apostrophes with straight quotes
    json_str = json_str.replace("’", "'")
    # Replace any incorrect quotes (e.g., mixed single and double quotes)
    json_str = json_str.replace("“", "\"").replace("”", "\"").replace("‘", "\"").replace("’", "\"")
    # Add escaping for quotes within the strings
```

```
    json_str = json_str.replace("'you didn"t'", "'you didn\'t'")
    return json_str

def getVideoSearchQueriesTimed(script,captions_timed):
    end = captions_timed[-1][0][1]
    try:

        out = [[[0,0],""]]
        while out[-1][0][1] != end:
            content = call_OpenAI(script,captions_timed).replace("'","")
            try:
                out = json.loads(content)
            except Exception as e:
                print("content: \n", content, "\n\n")
                print(e)
                content = fix_json(content.replace("```json", "").replace("```", ""))
                out = json.loads(content)
        return out
    except Exception as e:
        print("error in response",e)

    return None

def call_OpenAI(script,captions_timed):
    user_content = """Script: {}
Timed Captions:{}
""".format(script,"".join(map(str,captions_timed)))
    print("Content", user_content)

    response = client.chat.completions.create(
        model= model,
```

```
        temperature=1,
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": user_content}
        ]
    )

    text = response.choices[0].message.content.strip()
    text = re.sub('\s+', ' ', text)
    print("Text", text)
    log_response(LOG_TYPE_GPT,script,text)
    return text

def merge_empty_intervals(segments):
    merged = []
    i = 0
    while i < len(segments):
        interval, url = segments[i]
        if url is None:
            # Find consecutive None intervals
            j = i + 1
            while j < len(segments) and segments[j][1] is None:
                j += 1

            # Merge consecutive None intervals with the previous valid URL
            if i > 0:
                prev_interval, prev_url = merged[-1]
                if prev_url is not None and prev_interval[1] == interval[0]:
                    merged[-1] = [[prev_interval[0], segments[j-1][0][1]], prev_url]
                else:
                    merged.append([interval, prev_url])
```

```
        else:
            merged.append([interval, None])

        i = j
    else:
        merged.append([interval, url])
        i += 1

return merged
```

utils.py

```
import os
from datetime import datetime
import json

# Log types
LOG_TYPE_GPT = "GPT"
LOG_TYPE_PEXEL = "PEXEL"

# log directory paths
DIRECTORY_LOG_GPT = ".logs/gpt_logs"
DIRECTORY_LOG_PEXEL = ".logs/pexel_logs"

# method to log response from pexel and openai
def log_response(log_type, query,response):
    log_entry = {
        "query": query,
        "response": response,
        "timestamp": datetime.now().isoformat()
    }
    if log_type == LOG_TYPE_GPT:
        if not os.path.exists(DIRECTORY_LOG_GPT):
            os.makedirs(DIRECTORY_LOG_GPT)
        filename = '{}_gpt3.txt'.format(datetime.now().strftime("%Y%m%d_%H%M%S"))
        filepath = os.path.join(DIRECTORY_LOG_GPT, filename)
        with open(filepath, "w") as outfile:
            outfile.write(json.dumps(log_entry) + '\n')

    if log_type == LOG_TYPE_PEXEL:
        if not os.path.exists(DIRECTORY_LOG_PEXEL):
            os.makedirs(DIRECTORY_LOG_PEXEL)
```

```
filename = '{}_pexel.txt'.format(datetime.now().strftime("%Y%m%d_%H%M%S"))
filepath = os.path.join(DIRECTORY_LOG_PEXEL, filename)
with open(filepath, "w") as outfile:
    outfile.write(json.dumps(log_entry) + '\n')
```

5.2 Architectural Components

- **Frontend (UI Layer)**: Built with Streamlit for stage-wise interaction — Script, Voiceover, and Video Generation.
- **Controller Layer**: Manages session state, workflow logic, and integrates AI services.
- **Script Generator**: Uses ChatGPT via OpenAI API to create structured scripts.
- **Voiceover Generator**: Converts script to speech using EdgeTTS with multiple voice options.
- **Caption Generator**: Uses Whisper to generate time-aligned subtitles from audio.
- **Video Asset Manager**: Retrieves stock footage using Pexels API and aligns it with captions.
- **Rendering Engine**: Combines audio, captions, and video using MoviePy and FFmpeg.
- **Configuration & Storage**: Handles audio/video paths, user preferences, and API constants.

5.3 Feature Extraction

- The system performs implicit feature extraction at multiple stages to enable intelligent video generation:
- Text Feature Extraction: Semantic keywords and context are extracted from user-generated scripts to generate search queries for relevant video content.
- Audio Feature Extraction: The Whisper model analyzes the audio waveform to extract timing and phonetic features, enabling accurate generation of time-aligned captions.
- Temporal Feature Extraction: Caption timestamps and voiceover pacing are used to segment the video timeline, allowing precise alignment of video clips with the narration.
- Visual Feature Matching (via Pexels API): Extracted keywords and scene descriptions are used to retrieve stock footage that visually matches the script context.
- These features are critical for automating content alignment, improving relevance, and ensuring a seamless viewing experience.

5.4 Packages / Libraries Used

- Streamlit is used to create a clean and interactive user interface for each stage of the app. OpenAI (ChatGPT API) is used to generate structured scripts from user-inputted topics using natural language processing.
- EdgeTTS is used for converting text scripts into natural-sounding voiceovers with customizable voices.
- Whisper is implemented to generate accurate, timed subtitles from the generated audio using speech recognition.
- MoviePy is used for video editing tasks such as merging video clips, adding audio, and overlaying captions.
- FFmpeg works as a backend tool for MoviePy to process, convert, and render high-quality video outputs.
- NumPy helps manage numerical data during the captioning and rendering stages.
- Pandas is used for organizing and handling structured caption and script data.
- GitHub is used for version control, collaboration, and codebase management.
- Pexels API is used to retrieve relevant royalty-free video clips based on search terms generated from script and captions.

5.5 Output Screens

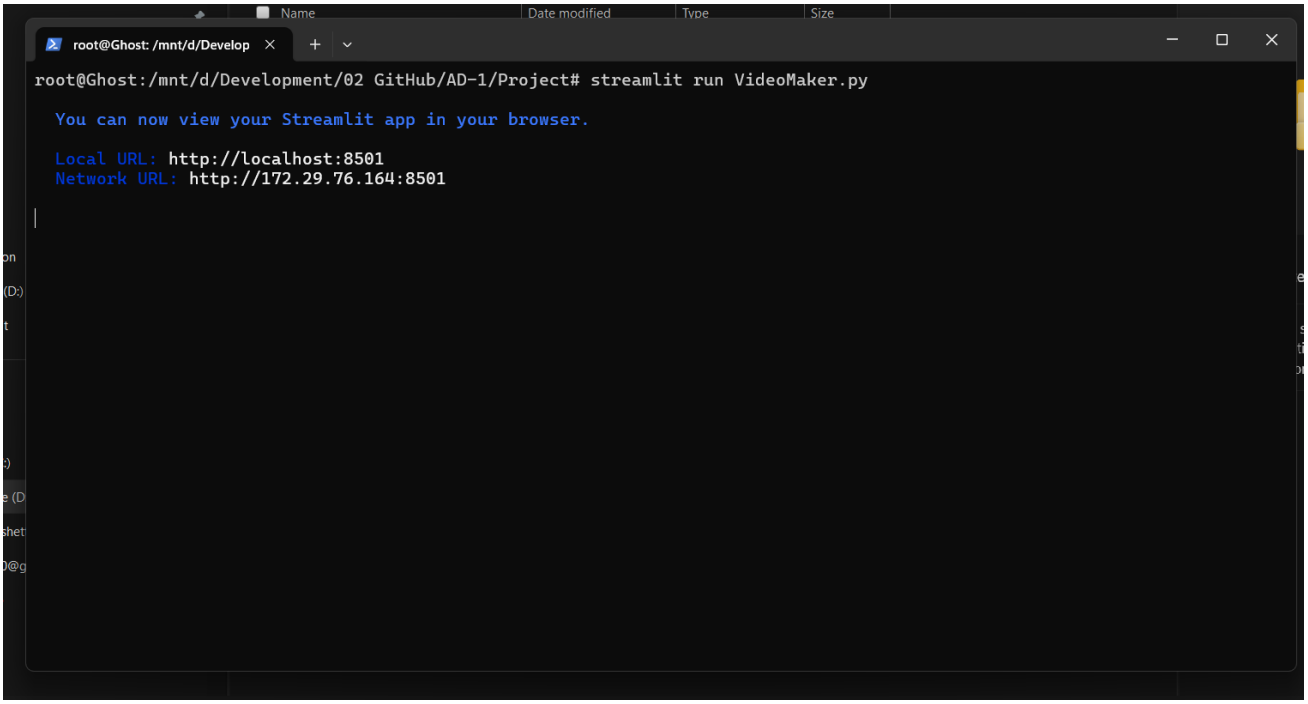


Fig 5.5(a) Initialization of application

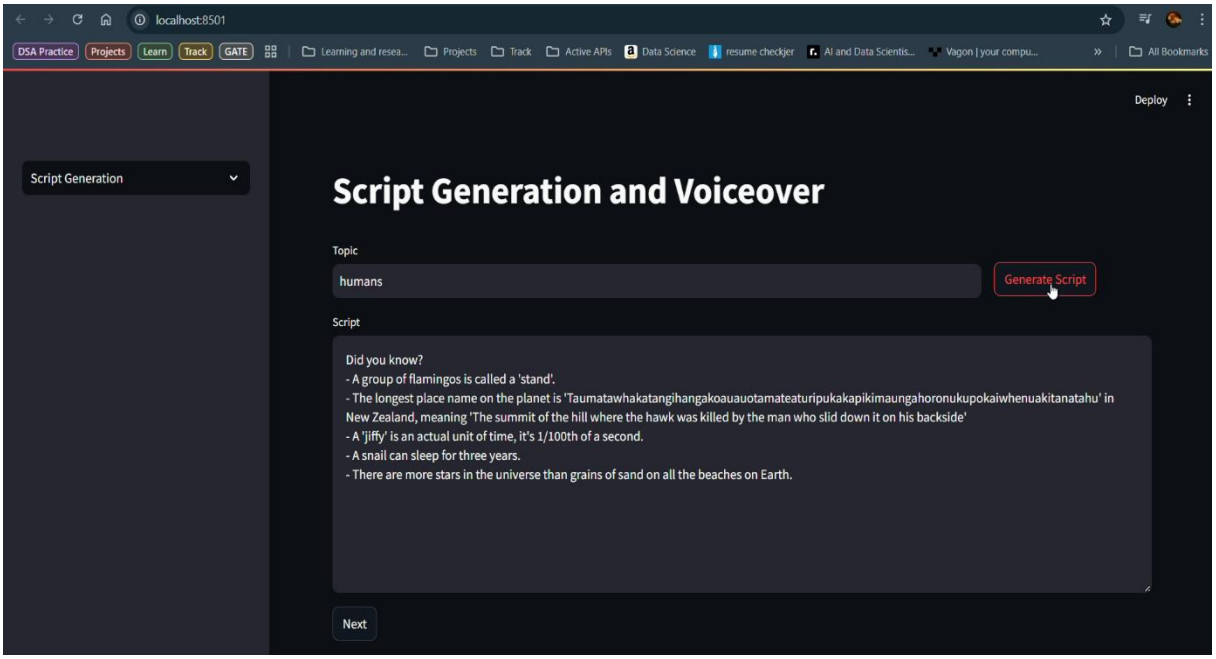


Fig 5.5(b) Script Generation phase

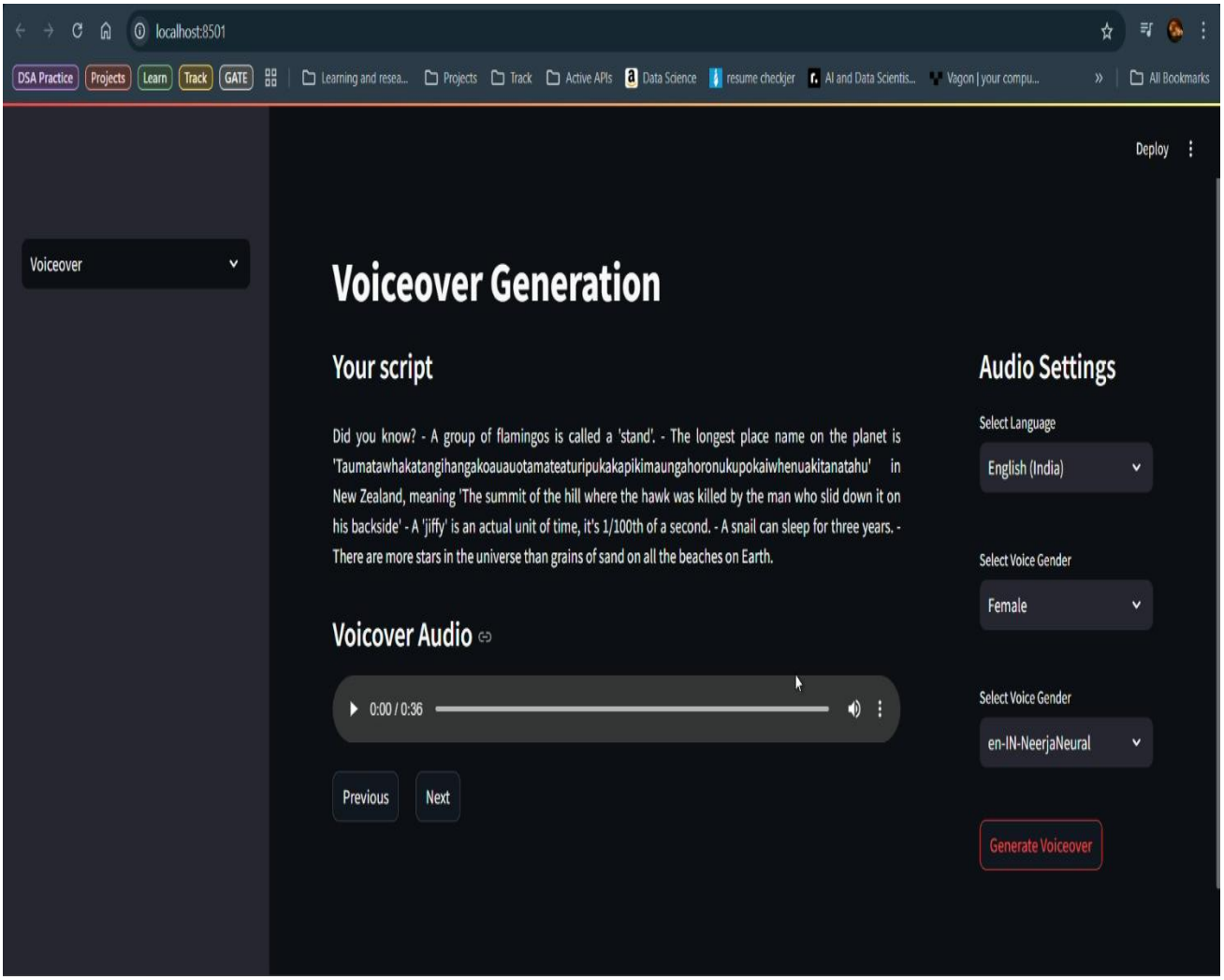


Fig 5.5(c) Voiceover Genration Phase

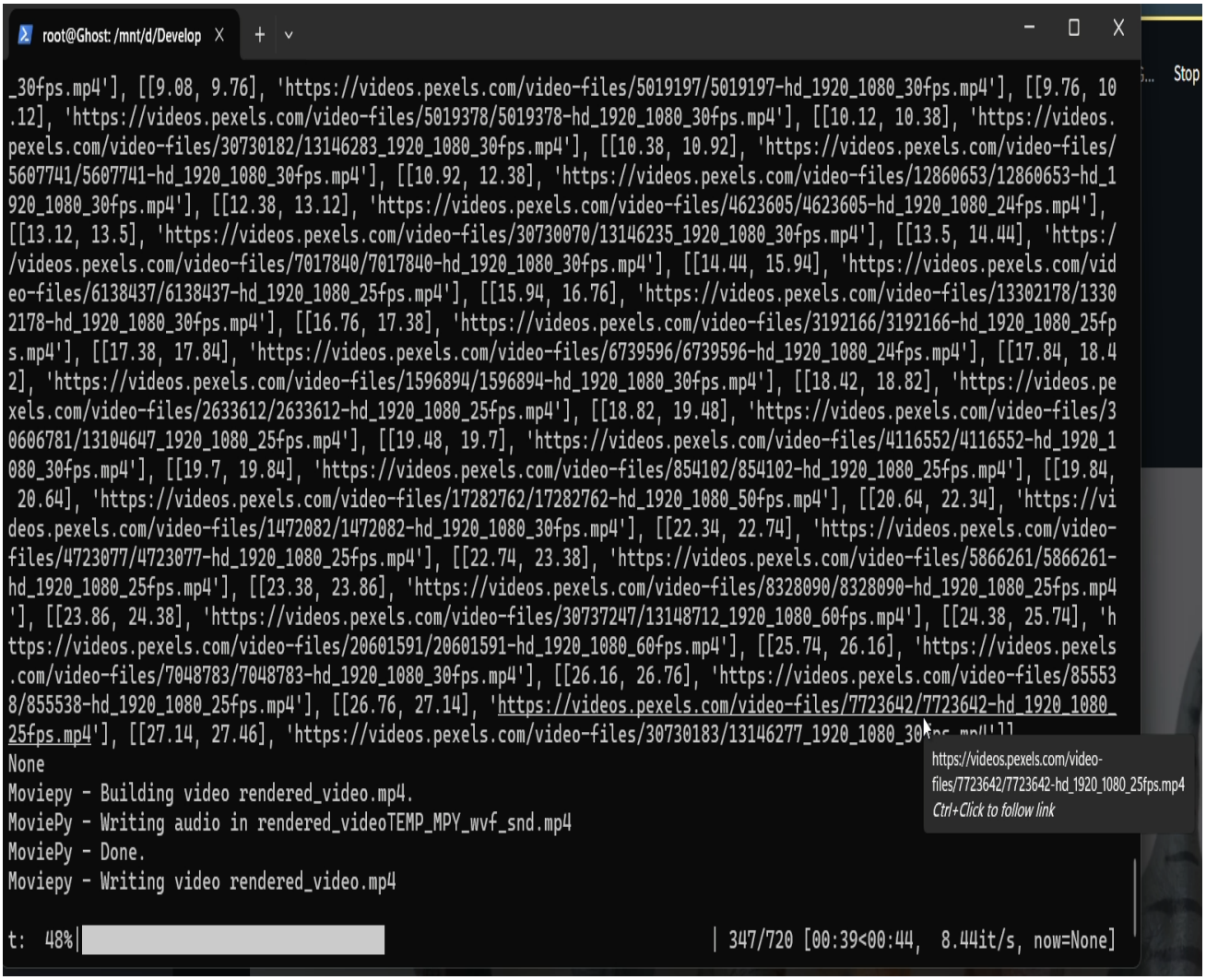


Fig 5.5(d) Video Rendering Phase Shell Interface

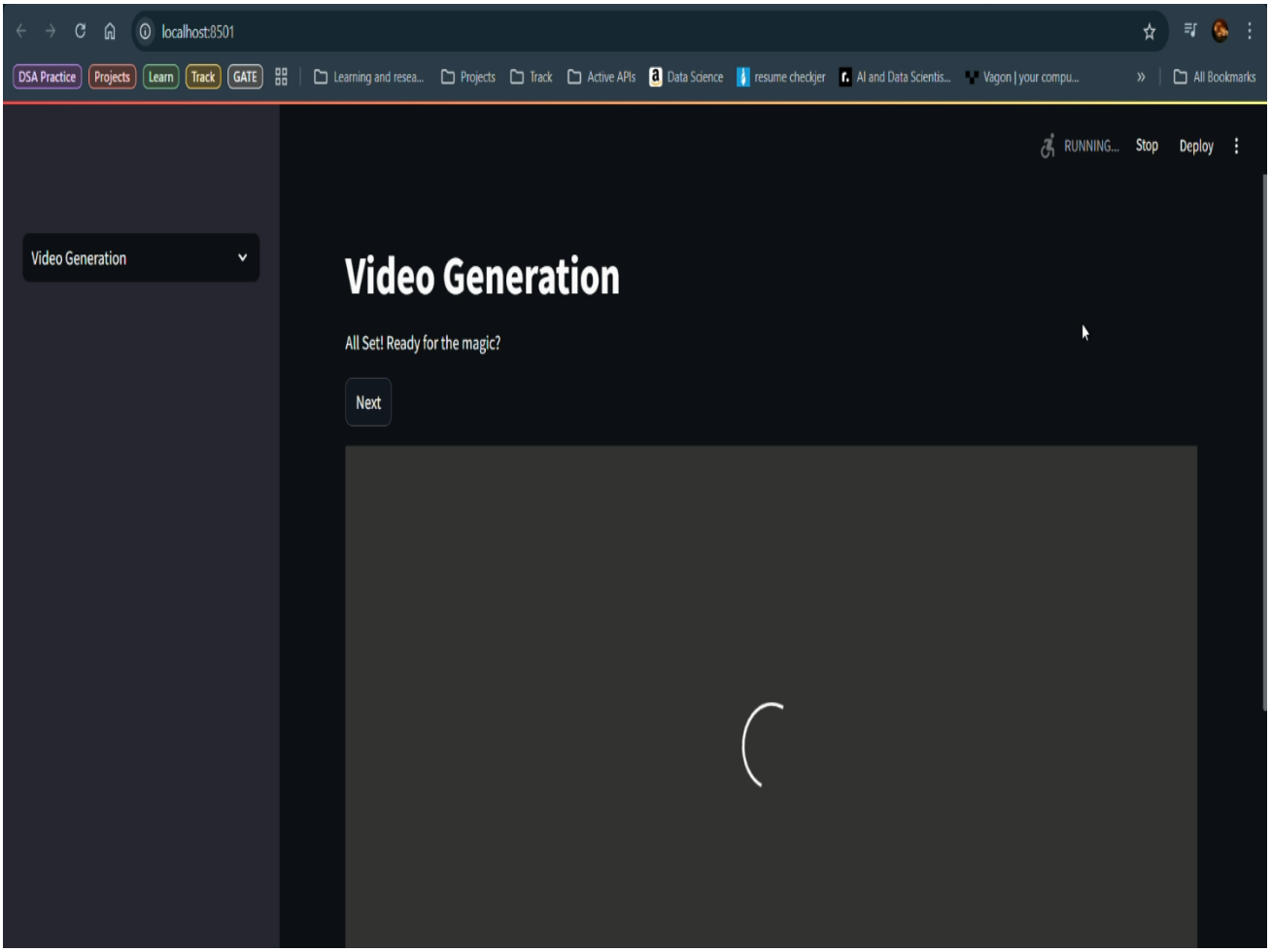


Fig 5.5(e) Video Generation Phase on GUI

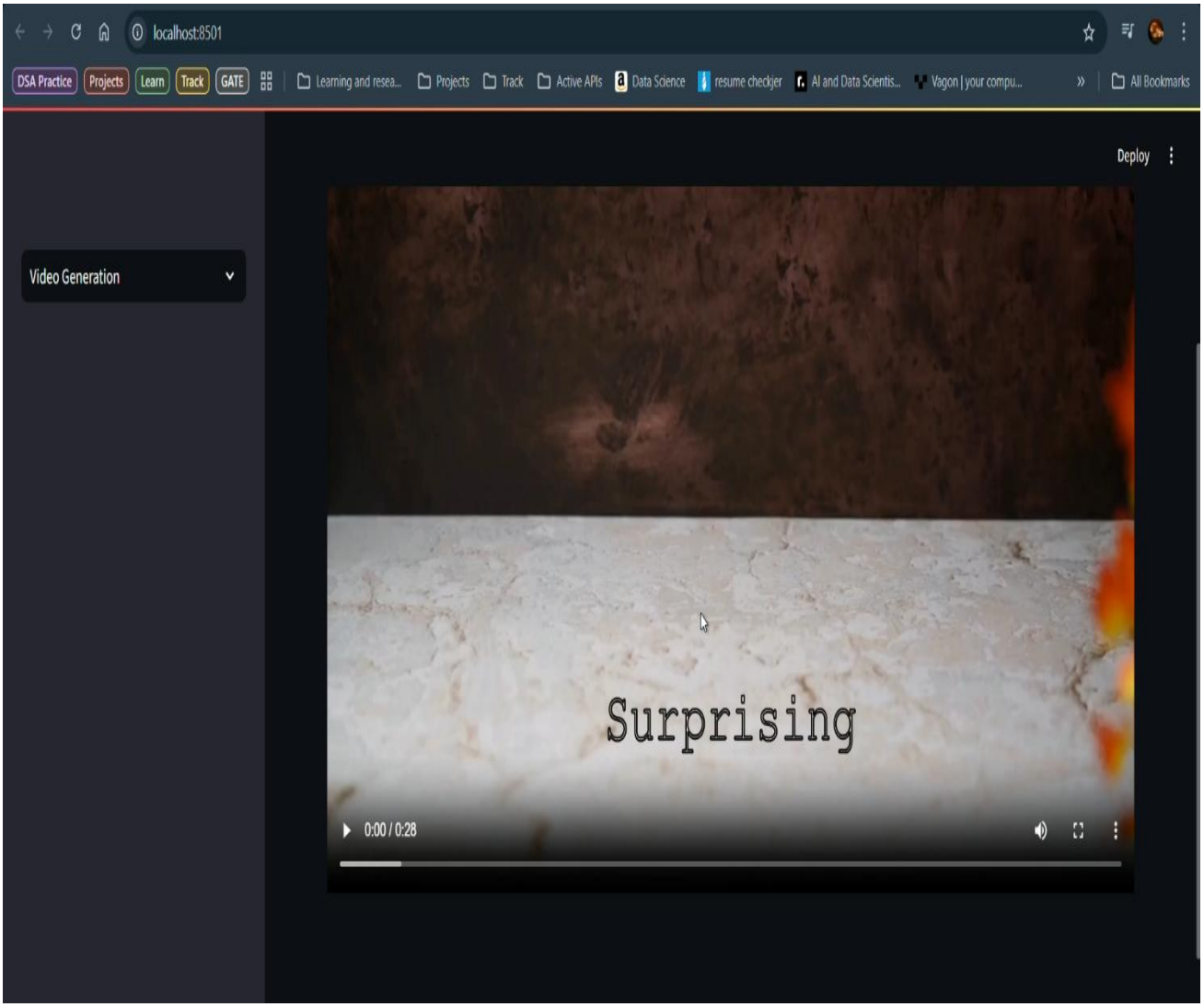


Fig 5.5(f) Generated Video Successfully Displayed to The User

CHAPTER 6
SYSTEM TESTING

6.1 Test Cases

The following test cases were implemented and successfully passed during testing:•

Input Validation in Script Generation:

Verified that an empty topic input displays a warning message prompting the user to enter valid input.

Tested whitespace-only input and confirmed the same validation behavior.

Script Generation Functionality:

Ensured that clicking "Generate Script" with valid input calls the generate_script() function and displays the output in the text area.

Confirmed that the script is stored in session state for further processing.

Stage Transition – Script to Voiceover:

Verified that clicking "Next" on the Script Generation stage transitions correctly to the Voiceover stage only if a script is present.

Confirmed that the script is displayed on the voiceover page.

Voiceover Generation with Valid Input:

Tested voice selection based on language and gender dropdowns.

Ensured that clicking "Generate Voiceover" triggers audio generation and plays the resulting file.

Audio Playback Validation:

Confirmed that the generated audio_tts.wav plays without errors in the Streamlit audio player.

Stage Transition – Voiceover to Video Generation:

Validated that "Next" moves to the Video Generation stage only if audio was successfully generated.

Ensured state consistency for audio and script values across transitions.

Video Generation Trigger:

Verified that clicking "Next" in Video Generation calls all required functions: generate_timed_captions, getVideoSearchQueriesTimed, generate_video_url, and get_output_media.

Checked that the video is rendered and displayed if all previous stages completed successfully.

Incomplete Workflow Handling:

Confirmed that the app shows a warning if the user attempts to generate a video without completing the script or voiceover stages.

6.2 Results and Discussions

The AI-Driven Automated Video Generator underwent thorough testing and evaluation across all functional modules to validate its performance, stability, and real-world applicability. The system was assessed through end-to-end user flows, simulating diverse usage scenarios to ensure that each component delivered a cohesive, reliable, and intuitive experience from start to finish.

Session-Based Navigation and Workflow Integrity

A session-based navigation mechanism was implemented using Streamlit's session_state, which played a critical role in maintaining continuity across the three main stages of the application:

1. Script Generation
2. Voiceover Creation
3. Video Compilation

This approach ensured that the user’s progress, data inputs, and selections were preserved throughout the session. Even during stage transitions or content updates, the system effectively retained state without any loss of data or reinitialization issues. This significantly contributed to an uninterrupted and user-friendly workflow, allowing users to navigate freely between stages without having to re-enter information or restart the process.

Robust Input Validation and User Feedback

The system incorporated rigorous input validation mechanisms to handle user prompts with care and precision. Specifically:

- Blank or whitespace-only prompts were detected and blocked from submission.
- Informative and context-aware warning messages were displayed, keeping the user informed and reducing friction during interaction.
- Edge cases such as extremely short or nonsensical input were gracefully handled, maintaining the quality and relevance of downstream outputs.

This validation layer enhanced the reliability of the overall system while promoting a more guided and supportive user experience.

Voiceover Generation via EdgeTTS

The voice synthesis component was powered by EdgeTTS, a high-performance text-to-speech engine. Testing across a variety of scripts and voice configurations demonstrated the following:

- The output audio was consistently natural, clear, and expressive, mimicking human speech with high fidelity.
- Multiple languages and voice personas (male/female, regional accents, etc.) were tested, all delivering accurate and pleasant results.
- Playback was seamlessly integrated via Streamlit's built-in audio player, ensuring users could instantly preview generated voiceovers.
- Latency was minimal, even for longer scripts, validating the system's readiness for production-level usage.

This module significantly enriched the video content, adding a professional touch to the automatically generated narrations.

Accurate and Aligned Captioning via Whisper

Subtitles were generated using OpenAI's Whisper model, known for its state-of-the-art speech recognition capabilities. Key findings from this module include:

- Captions were highly accurate, with correct transcription even in cases of long or complex script content.
- Time alignment between the voiceover and the captions was well-maintained, ensuring synchrony and readability.
- The integration of subtitles not only improved accessibility for hearing-impaired users but also enhanced the clarity and impact of the final video.
- No significant desynchronization or misinterpretation issues were observed across multiple test runs.

Automated Video Compilation and Rendering

The final video assembly was executed via a modular pipeline incorporating:

- Keyword Extraction from generated scripts
- Video Retrieval using the Pexels API
- Interval Stitching to merge or trim empty video segments
- Rendering and Export via MoviePy and FFmpeg

This automated pipeline performed with high consistency, producing videos that were:

- Visually engaging, with relevant clips matching the narration context
- Synchronized across visuals, captions, and audio
- Rendered in suitable formats with optimized resolution and bitrate settings

The system also managed exception handling during video merging, such as missing footage or formatting discrepancies, ensuring robustness and minimal downtime.

The AI-Driven Automated Video Generator proved to be a fully functional, stable, and user-centric solution for end-to-end content automation. Each module was tested under realistic conditions, and the integrated system demonstrated strong cohesion across the workflow. The platform's architecture enables scalable enhancements, such as additional voice models, multilingual support, custom branding, or export templates, making it suitable for use in education, content marketing, and social media automation.

By effectively bridging AI-based scripting, voice synthesis, captioning, and video creation, the system lays the groundwork for democratizing high-quality video content production with minimal human effort.

6.3 Performance Evaluation

The system was also evaluated for performance to ensure responsiveness, low latency, and efficient resource usage during execution. Below are the observed metrics:

• Script Generation Speed:

Average response time for generating a full-length script using the OpenAI ChatGPT API was between **2–3 seconds**, depending on internet speed and token length.

• Voiceover Generation Time:

The time taken to convert a one-minute script into speech using EdgeTTS was between **4–6 seconds** on average. Output quality was consistent across different voice selections.

• Captioning and Video Rendering Time:

The caption generation process completed within **5–7 seconds** for standard audio files. The full video rendering pipeline — including merging audio, background footage, and captions — was completed within **1.5 to 2 minutes**, depending on system hardware and the number of video segments.

• Responsiveness:

The **Streamlit interface** remained responsive during all stages, including long-running tasks like video rendering. No UI freezing, lag, or session timeout issues were encountered. Button clicks, input fields, and transitions worked smoothly under both single and multiple execution cycles.

These performance outcomes confirm that the system is optimized for typical hardware setups and supports efficient, real-time content generation.

CHAPTER 7
CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

The **AI-Driven Automated Video Generator for Content Creation** represents a comprehensive solution to one of the most time-consuming and resource-intensive challenges in digital media production — transforming ideas into complete, videos with minimal manual intervention. By integrating multiple AI technologies into a unified platform, this system significantly **reduces the need for complex software tools** and human resources typically involved in **scripting, voiceover, captioning, and video editing.**

The application is designed to operate in three intuitive stages: script generation, voiceover synthesis, and video compilation. **Each stage is driven by advanced AI models and APIs** — including **ChatGPT** for **natural language generation, EdgeTTS** for lifelike speech synthesis, and Whisper for accurate, timed captioning. The final video assembly, powered by **MoviePy** and **FFmpeg**, intelligently combines the script, narration, and visuals retrieved from the **Pexels API** to produce coherent, visually appealing video content that can be used across **educational, marketing, and social platforms.**

One of the standout features of this system is its **simplicity and accessibility**. Developed using **Streamlit**, the interface is user-friendly and tailored to both technical and non-technical users, making it possible for **individuals with no prior video editing experience to generate high-quality content with just a few inputs**. Additionally, the **modular architecture** ensures that each component functions independently, allowing for **future enhancements** such **as multilingual support, alternative AI models** and integration with additional media libraries.

The project's architecture reflects best practices in **AI integration** and software design. With the use of session state handling, dynamic input management, and asynchronous audio processing, the platform delivers a responsive and efficient experience. Resource optimization has been taken into consideration, making it viable to run on systems with moderate hardware configurations while still ensuring real-time feedback and media generation.

In a world where content is increasingly consumed in multimedia formats, this system offers a scalable, intelligent alternative to traditional content creation workflows. It empowers educators to generate informative lessons, marketers to produce branded videos, and content creators to transform ideas into shareable stories — all in a matter of minutes. Beyond its current functionality, the platform serves as a strong foundation for future research and development in generative multimedia, AI-powered storytelling, and automated creative tools.

In summary, the AI-Driven Automated Video Generator exemplifies the fusion of artificial intelligence and media technology in a practical, impactful, and extensible application. It not only enhances productivity and creativity but also paves the way for the next generation of intelligent content creation systems.

CHAPTER 8
REFERENCES

- **AI-Powered Content Creation** – A study on **Generative AI in multimedia production**.
- **Text-to-Video AI Models** – Research on **Natural Language Processing (NLP) and Deep Learning** for automated video synthesis.
- **Ethical AI Guidelines for Content Creation** – IEEE standards ensuring AI-generated media complies with ethical AI guidelines.

LIST OF FIGURES

Fig. No	Figure Title	Page no.
4.1	Architecture Diagram	9
4.2	Dataflow Diagram	10
4.3.1	Class Diagram	11
4.3.2	Use Case Diagram	12
4.3.3	Sequential Diagram	13
4.3.4	Activity Diagram	14

LIST OF ABBREVIATIONS

S. N	ABBREVIATION	FULL FORM
1	LLM	Large Language Model
2	API	Application Programming Interface
3	JSON	JavaScript Object Notation
4	CLI	Command Line Interface
5	NLP	Natural Language Processing
6	TTS	Text-to-Speech
7	ASR	Automatic Speech Recognition
8	UI	User Interface
9	GPT	Generative Pre-trained Transformer
10	SRS	Software Requirements Specification
11	DFD	Data Flow Diagram
12	FFmpeg	Fast Forward MPEG
13	SSD	Solid State Drive
14	RAM	Random Access Memory

LIST OF TABLES

S. No	Table Name	Page No
1.	Input Validation in Script Generation:	52
2.	Script Generation Functionality:	53
3.	Stage Transition – Script to Voiceover:	53
4.	Voiceover Generation with Valid Input:	54
5.	Audio Playback Validation:	54
6.	Stage Transition – Voiceover to Video Generation:	55
7.	Video Generation Trigger:	55